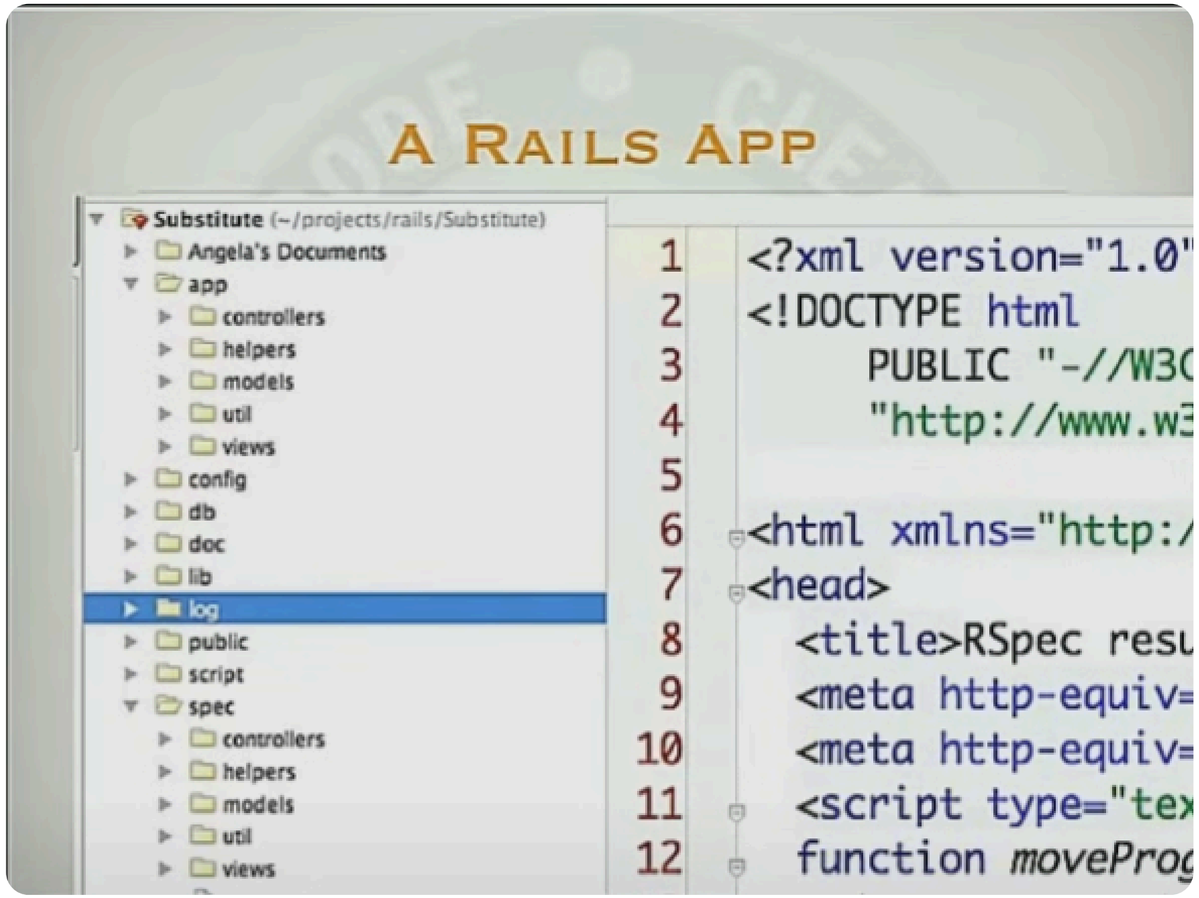


Laboratorio di Informatica

DDD & Clean Architecture

Luca Pulga
Istituto Tecnico Tecnologico Blaise Pascal
luca.pulga@ispascalcomandini.it

WHY CLEAN ARCHITECTURE

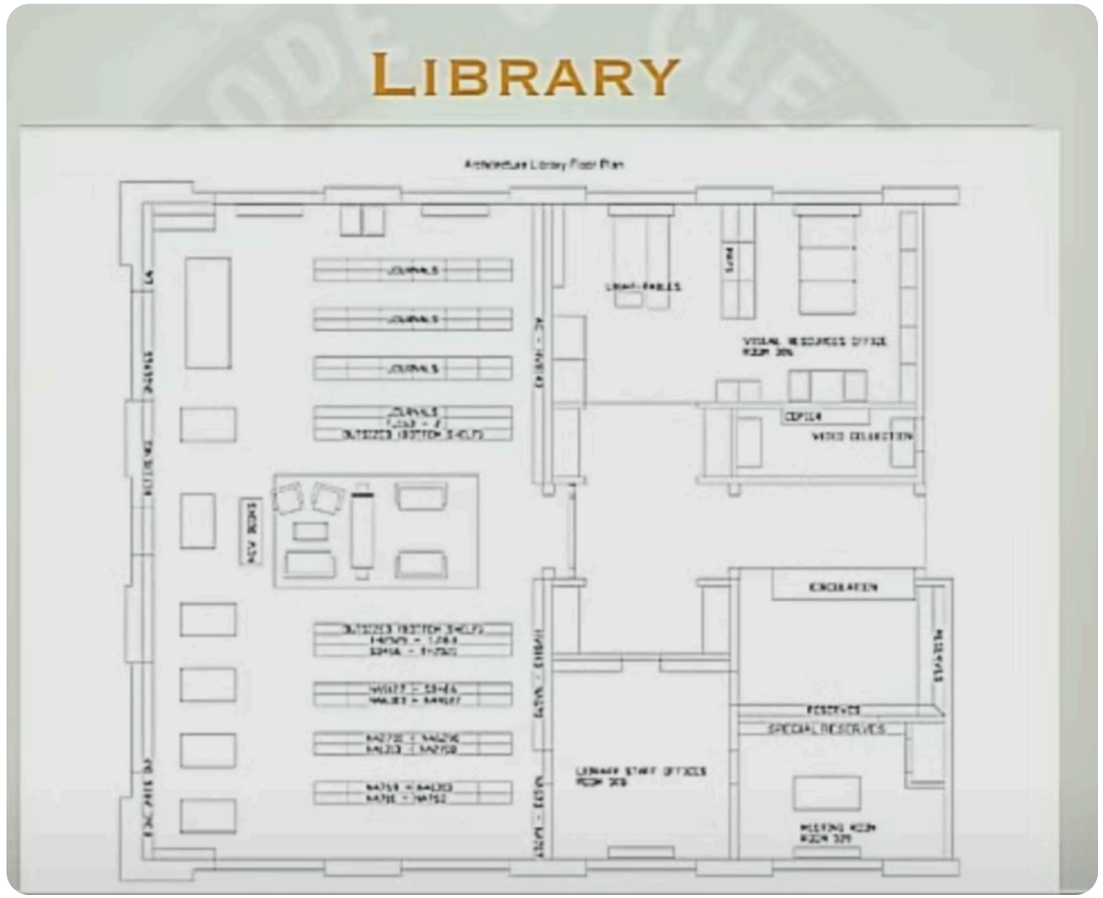


"This project is not communicating to me **what** its **purpose is**, it is simply the template of a **generic**, **well-structured project,** but it is **not communicating** to me what it is doing."

U.Bob

BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

WHY CLEAN ARCHITECTURE

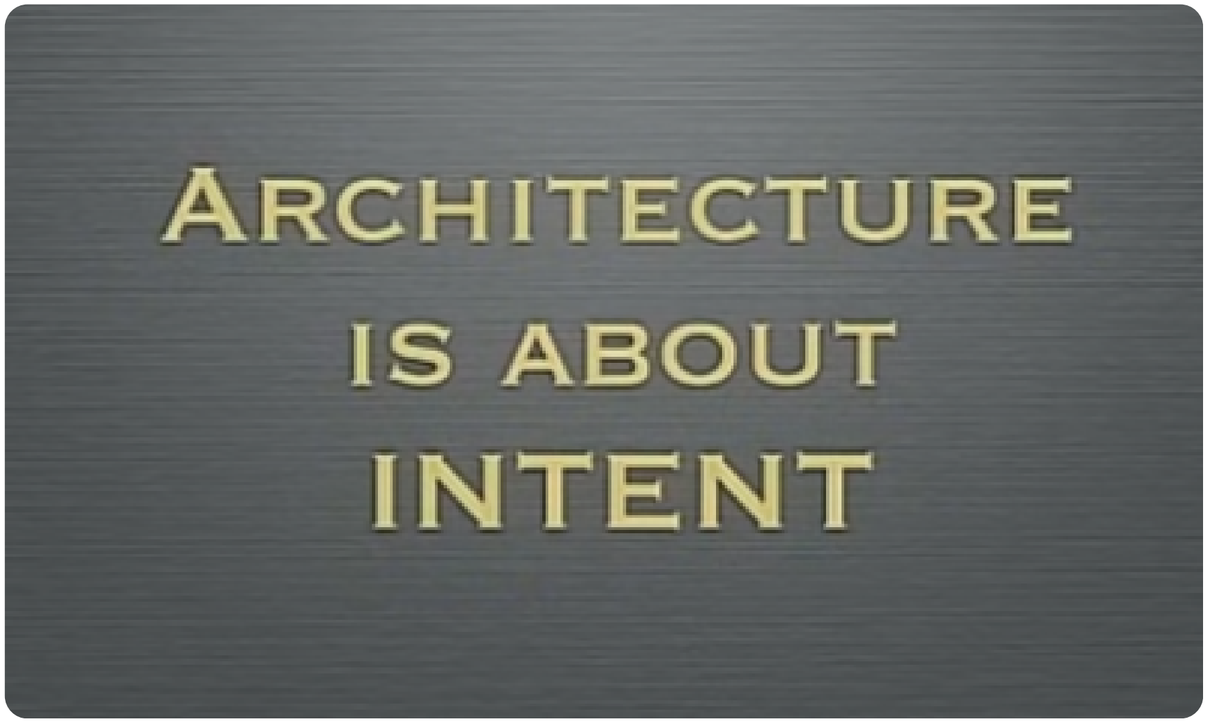


"This project is **communicating to me** what its **purpose** is, it is simply the well-structured design of a library and it is communicating to me what it is.
It does not tell us the material it is made of, the framework that is being used to build it, but **it specifies the intent perfectly.**"

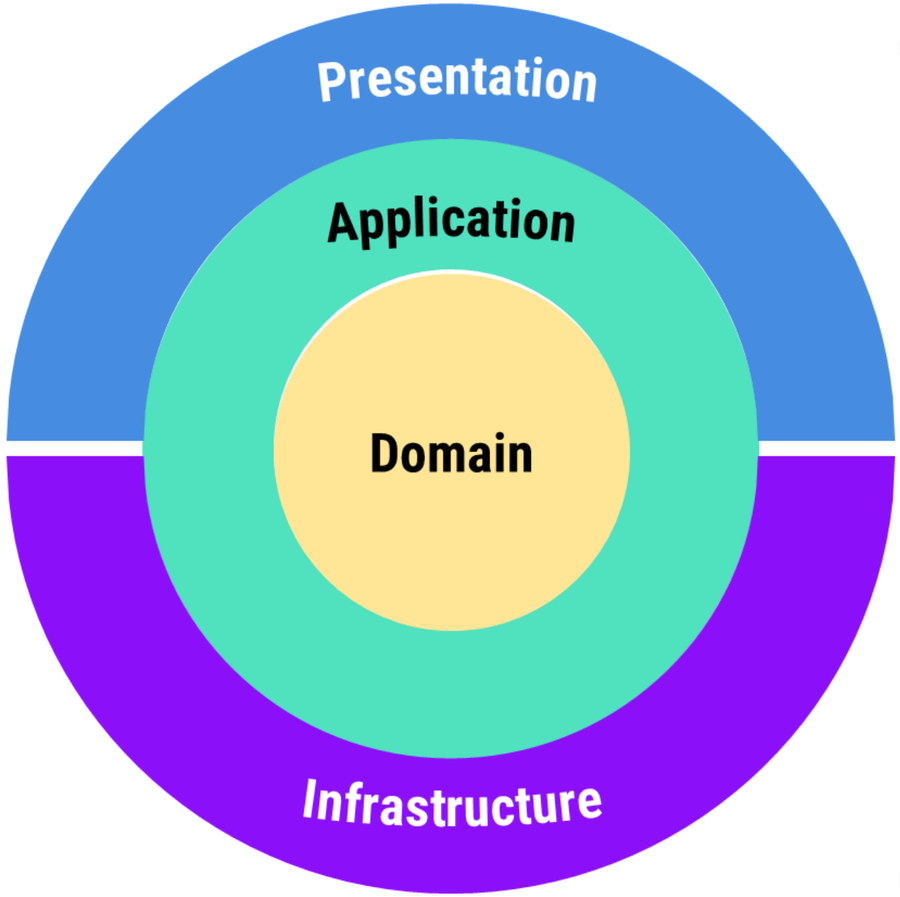
U.Bob

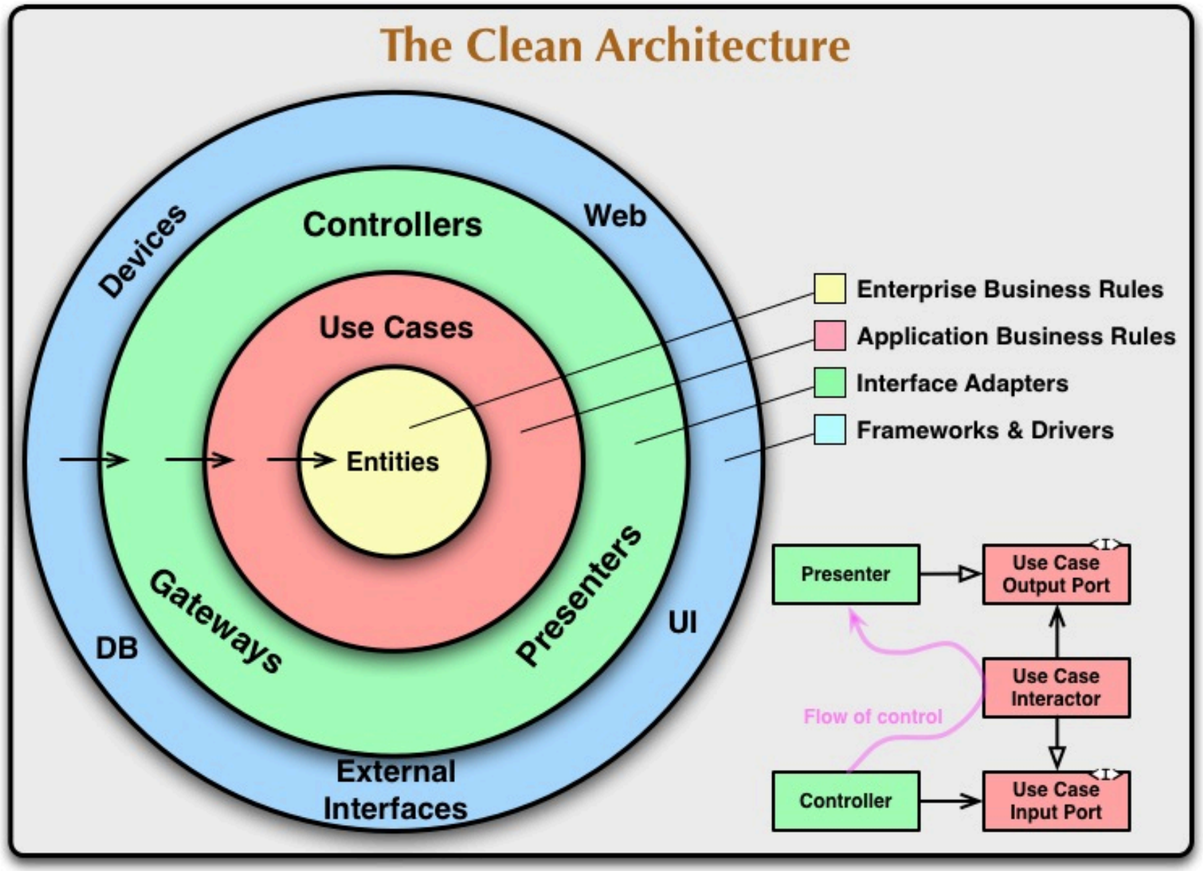
BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

WHY CLEAN ARCHITECTURE



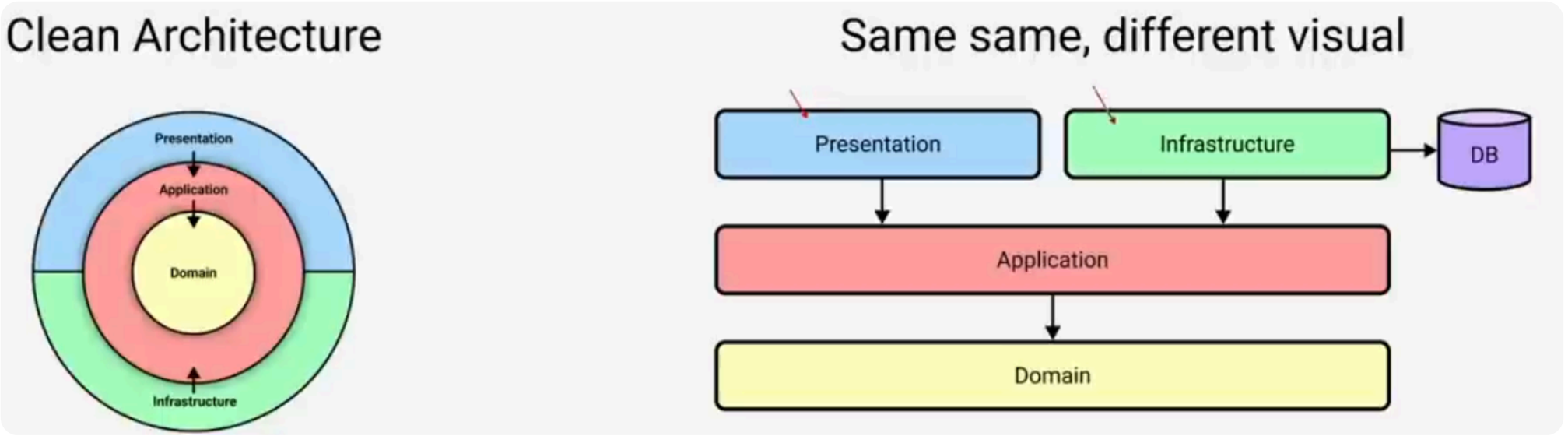
WHY CLEAN ARCHITECTURE





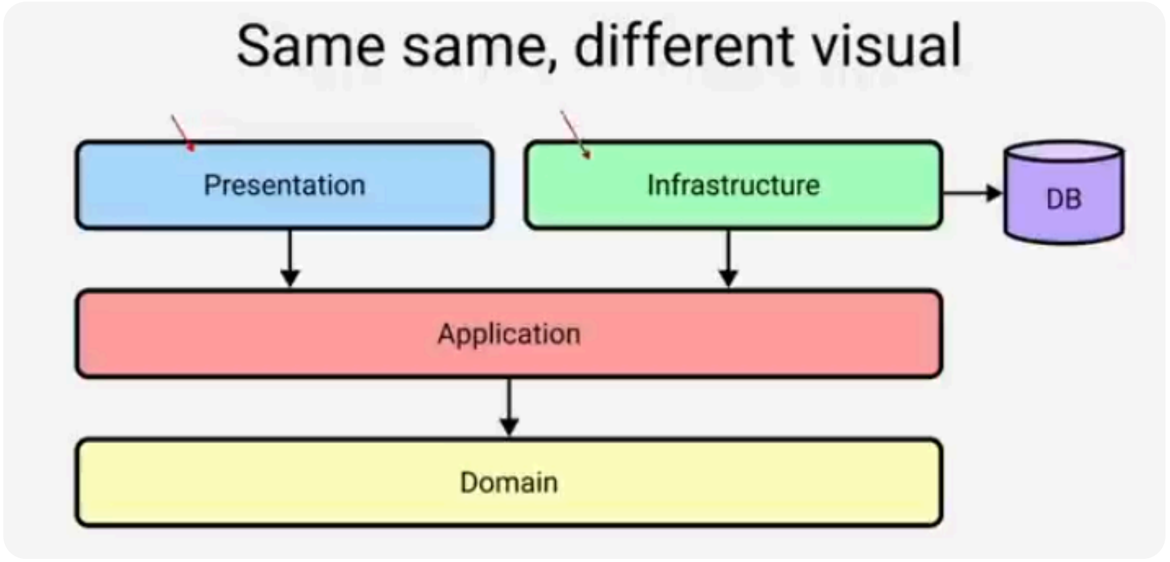
BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

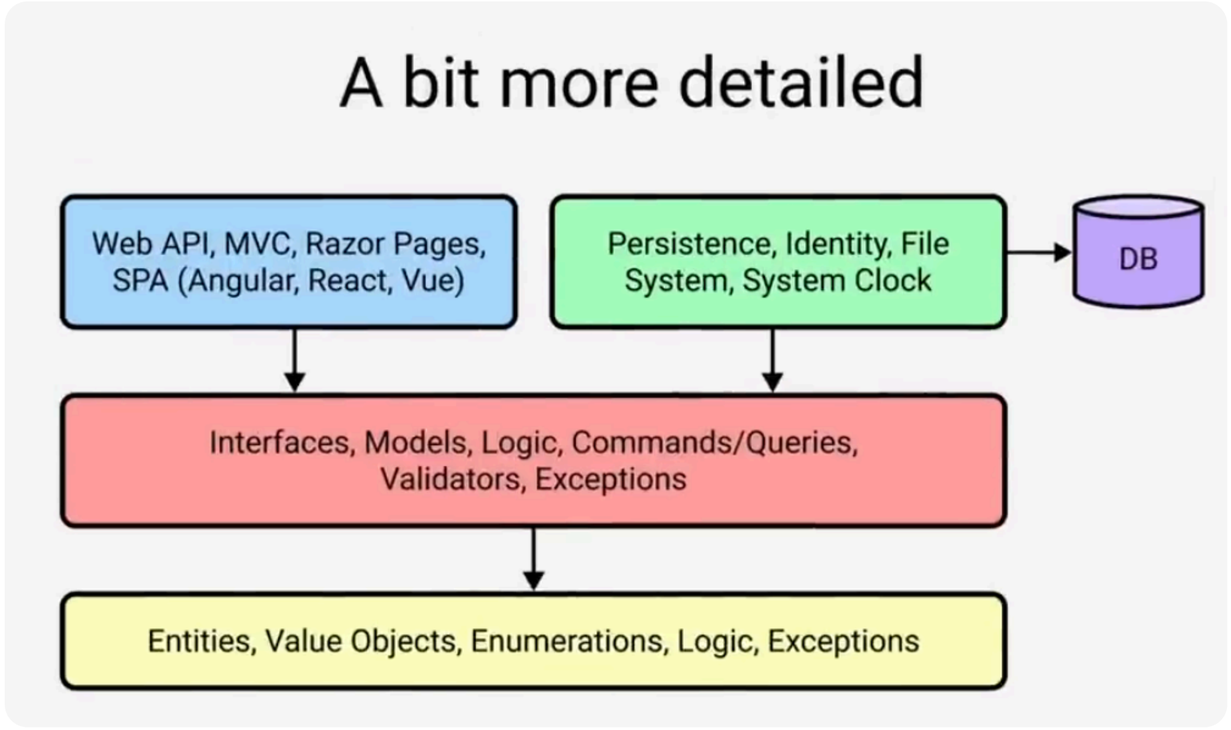
WHY CLEAN ARCHITECTURE



BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

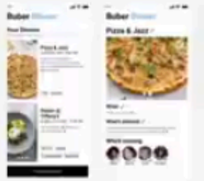
WHY CLEAN ARCHITECTURE





WHY CLEAN ARCHITECTURE

Final Result w/ fancy words



- **Libraries**
  - Mapster
- **Concepts/Patterns**
  - Mediator

- **Libraries**
  - MediatR
    - Requests
    - Notifications
    - Behaviors
  - FluentValidation
- **Concepts/Patterns**
  - CQRS
  - Repository
  - Unit of Work

- **Libraries**
  - EF Core
  - BCrypt
- **Concepts/ Patterns**
  - ORM

- **Libraries**
  - ErrorOr
- **Concepts/Patterns**
  - Aggregates
  - Aggregate Roots
  - ValueObjects
  - Entities
  - Domain Errors
  - Domain Events

- **Some other fancy words**
  - Clean Architecture
  - Domain Driven Design
- **Fancy Tech**
  - Visual Studio Code
  - dotnet CLI

BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

VISUAL STUDIO: CLI PROJECT SETUP

1) crea cartella radice e solution

```
mkdir SmartHouse
cd SmartHouse
dotnet new sln -n BlaisePascal.SmartHouse
```

2) crea cartelle principali

```
mkdir src test docs
```

3) crea i progetti

```
dotnet new classlib -n BlaisePascal.SmartHouse.Domain -o src/BlaisePascal.SmartHouse.Domain -f net9.0
```

4) aggiungi i progetti alla solution, dentro la solution-folder "src"

```
dotnet sln add src/BlaisePascal.SmartHouse.Domain/BlaisePascal.SmartHouse.Domain.csproj --solution-folder src
```

VISUAL STUDIO: CLI PROJECT SETUP

5) crea i progetti di test (xUnit per test) dentro test/
dotnet new xunit -n BlaisePascal.SmartHouse.Domain.UnitTests -o test/BlaisePascal.SmartHouse.Domain.UnitTests -f net9.0

6) aggiungi i progetti test alla solution sotto la solution-folder "test"
dotnet sln add test/BlaisePascal.SmartHouse.Domain.UnitTests/BlaisePascal.SmartHouse.Domain.UnitTests.csproj --solution-folder test

7) aggiungi riferimenti tra progetti (es. Test -> Domain)
dotnet add test/BlaisePascal.SmartHouse.Application/BlaisePascal.SmartHouse.Domain.UnitTests.csproj reference src/BlaisePascal.SmartHouse.Domain/BlaisePascal.SmartHouse.Domain.csproj

VISUAL STUDIO: CLI PROJECT SETUP

8) collega test ai progetti da testare
dotnet add
test/BlaisePascal.SmartHouse.Domain.UnitTests/BlaisePascal.SmartHouse.Domain.UnitTests.csproj
reference src/BlaisePascal.SmartHouse.Domain/BlaisePascal.SmartHouse.Domain.csproj

9) restore, build e run tests
dotnet restore
(Scarica e prepara tutte le dipendenze NuGet necessarie per compilare il progetto/soluzione)

dotnet build
(Compila il codice e produce l'output (DLL, PDB, ecc.) per esecuzione o test)

dotnet test
(compila ed esegue tutti i test automatici in un progetto o soluzione .NET)

VISUAL STUDIO: GUI PROJECT SETUP

- Apri Visual Studio → File → New → Project → cerca **Blank Solution** (o "**Empty Solution**" o "**Soluzione Vuota**") e crea la **solution SmartHouse** in una cartella a tua scelta.
- Nel file system, crea le **cartelle src, test, docs** dentro la cartella della **solution** (puoi farlo anche da VS con **Add → New Folder** su livello solution, ma questo crea solo Solution Folder, non la cartella fisica; per il file system è comodo crearle manualmente).
- In Visual Studio, sul livello della solution fai **Add → New Project** e quando ti viene chiesto il percorso, seleziona .../**src** come cartella di destinazione e nomina il progetto **BlaisePascal.SmartHouse.Domain** (scegli template **Class Library**).
- **Per i test**: **Add → New Project → scegli xUnit Test Project** e salva i progetti nella cartella test, nominando **BlaisePascal.SmartHouse.Domain.UnitTests**, ecc.
- **Aggiungi reference tra progetti**: nella Solution Explorer, clic destro su un progetto → **Add → Project Reference**... e seleziona i progetti target (es. Test riferisce Domain).
- Per avere i file docs nella soluzione: crea i file docs/README.md, docs/architecture.md sul filesystem, poi in Visual Studio clic destro sulla solution → Add → Existing Item... e seleziona i markdown; verranno aggiunti come Solution Items.

BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

RECOMMENDED FOLDER STRUCTURE

Per un progetto C# basato su DDD, si consiglia di separare il codice in livelli principali:
Dominio, Applicazione e Infrastruttura.
La struttura delle cartelle per un progetto di questo tipo potrebbe essere simile alla seguente:

```
ProjectName/
|-- src/
|   |-- ProjectName.Domain/
|   |-- ProjectName.Application/
|   |-- ProjectName.Infrastructure/
|-- tests/
|   |-- ProjectName.Domain.Tests/
|   |-- ProjectName.Application.Tests/
|   |-- ProjectName.Infrastructure.Tests/
```

ProjectName.Domain: contiene il modello di dominio, incluse entità, oggetti valore ed eventi di dominio.
ProjectName.Application: include casi d'uso, comandi, query e i rispettivi gestori, concentrandosi sul coordinamento delle attività e sulla delega del lavoro ai livelli di dominio e infrastruttura.
ProjectName.Infrastructure: gestisce la persistenza, i servizi esterni e altre questioni tecniche.

RECOMMENDED FOLDER STRUCTURE

```
ProjectName/
|-- src/
|   |-- ProjectName.Domain/
|   |-- ProjectName.Application/
|   |-- ProjectName.Infrastructure/
|-- tests/
|   |-- ProjectName.Domain.Tests/
|   |-- ProjectName.Application.Tests/
|   |-- ProjectName.Infrastructure.Tests/
```

ProjectName.Domain: contiene il modello di dominio, incluse entità, oggetti valore ed eventi di dominio.

ProjectName.Application: include casi d'uso, comandi, query e i rispettivi gestori, concentrandosi sul coordinamento delle attività e sulla delega del lavoro ai livelli di dominio e infrastruttura.

ProjectName.Infrastructure: gestisce la persistenza, i servizi esterni e altre questioni tecniche.

RECOMMENDED FOLDER STRUCTURE

Per un progetto C# basato su DDD, si consiglia di separare il codice in livelli principali: **Dominio, Applicazione e Infrastruttura**.
La struttura delle cartelle per un progetto di questo tipo potrebbe essere simile alla seguente:

```
ProjectName.Domain/
|-- Entities/
|-- ValueObjects/
|-- DomainEvents/
|-- Repositories/
|-- Specifications/
```

Organizza le classi di dominio in base al loro tipo (ad esempio, **entities**, **object values**, **domain events**) e, se necessario, suddividile ulteriormente in base ai loro specifici sottocontesti di dominio.

RECOMMENDED FOLDER STRUCTURE

Per un progetto C# basato su DDD, si consiglia di separare il codice in livelli principali: **Dominio, Applicazione e Infrastruttura**.
La struttura delle cartelle per un progetto di questo tipo potrebbe essere simile alla seguente:

```
ProjectName.Domain/
|-- Entities/
|-- ValueObjects/
|-- DomainEvents/
|-- Repositories/
|-- Specifications/
```

Organizza le classi di dominio in base al loro tipo (ad esempio, **entities**, **object values**, **domain events**) e, se necessario, suddividile ulteriormente in base ai loro specifici sottocontesti di dominio.

```
ProjectName.Application/
|-- Commands/
|   |-- Handlers/
|-- Queries/
|   |-- Handlers/
|-- Services/
|-- Validators/
```

Raggruppa le classi del livello applicativo in base alla loro funzione, come **command**, **query** e relativi **handler**.
Includi anche servizi e validatori che si occupano di aspetti trasversali (**cross-cutting concerns**).

RECOMMENDED FOLDER STRUCTURE

Per un progetto C# basato su DDD, si consiglia di separare il codice in livelli principali: **Dominio, Applicazione e Infrastruttura**.
La struttura delle cartelle per un progetto di questo tipo potrebbe essere simile alla seguente:

```
ProjectName.Domain/
|-- Entities/
|-- ValueObjects/
|-- DomainEvents/
|-- Repositories/
|-- Specifications/
```

Organizza le classi di dominio in base al loro tipo (ad esempio, **entities**, **object values**, **domain events**) e, se necessario, suddividile ulteriormente in base ai loro specifici sottocontesti di dominio.

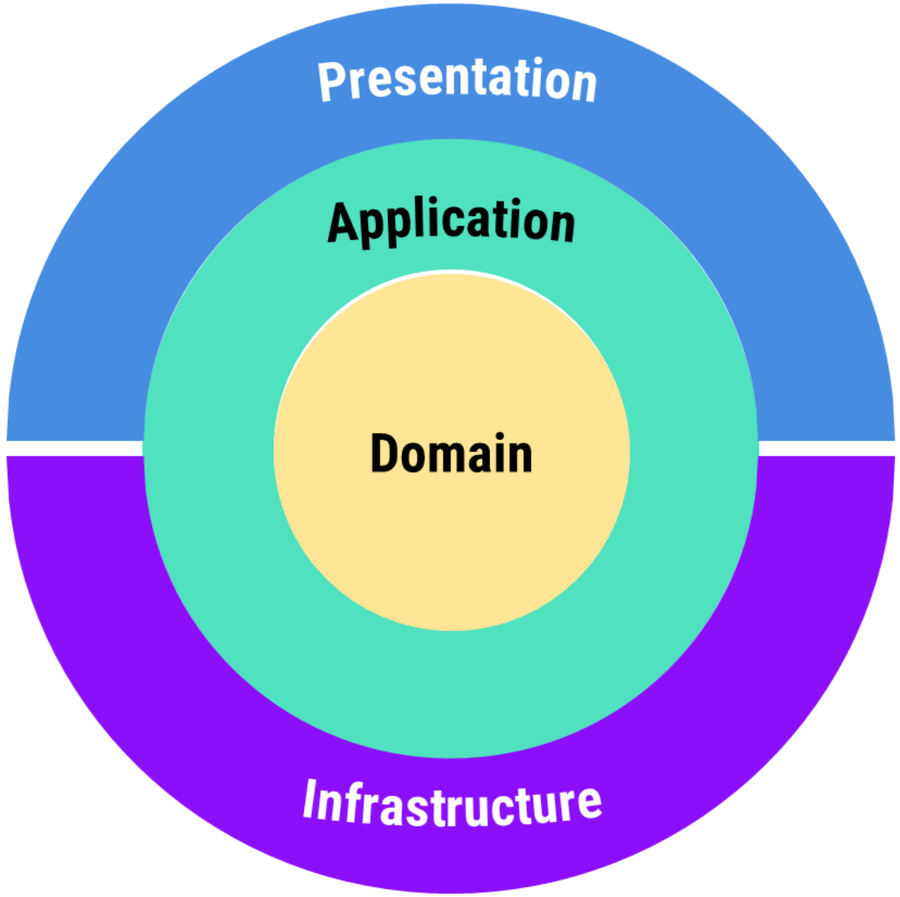
```
ProjectName.Application/
|-- Commands/
|   |-- Handlers/
|-- Queries/
|   |-- Handlers/
|-- Services/
|-- Validators/
```

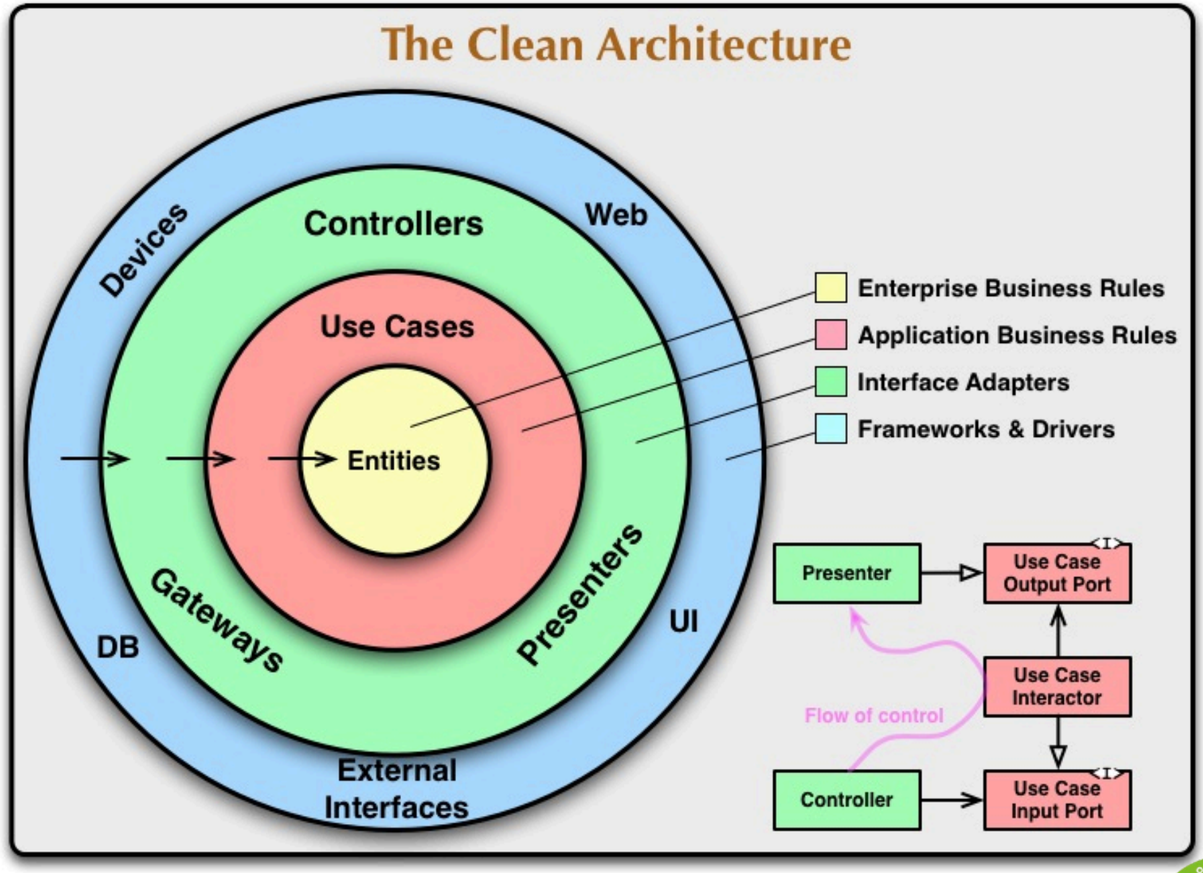
Raggruppa le classi del livello applicativo in base alla loro funzione, come **command**, **query** e relativi **handler**.
Includi anche servizi e validatori che si occupano di aspetti trasversali (**cross-cutting concerns**).

```
ProjectName.Infrastructure/
|-- Data/
|   |-- Repositories/
|   |-- Migrations/
|-- ExternalServices/
|   |-- Clients/
|-- Configuration/
|-- Extensions/
```

Organizza le classi del livello infrastrutturale in base al loro scopo, come **persistenza dei dati**, **integrazione di servizi esterni**, **configurazione e metodi di estensione.**

DOMAIN LAYER: BUILDING BLOCKS AND PATTERNS





BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

DOMAIN LAYER: BUILDING BLOCKS AND PATTERNS

Il **domain layer** è il cuore di un'applicazione basata sul DDD e contiene gli **elementi costitutivi** e i **modelli fondamentali** che **modellano il dominio del problema**.

Discuteremo gli elementi costitutivi chiave, tra cui **Entities**, **Value Objects** e **Domain Event**,

nonché **Design Patterns** rilevanti come **Factory Pattern**, **Repository Pattern** e **Specification Pattern**.

Nessuna dipendenza verso altri strati.

ENTITIES

Sono **oggetti con un'identità distinta** e comparabile che possono attraversare stati diversi.

Leggono proprietà e metodi comportamentali per poterne modificare lo stato interno.

Massima espressività.

ENTITIES

Le entità sono oggetti con un'identità univoca che può cambiare nel tempo.

Sono gli elementi costitutivi primari di un modello di dominio e **incapsulano sia lo stato che il comportamento.**

In C#, le entità sono in genere implementate come classi con una proprietà identificativa univoca (ad esempio un GUID o un numero intero) e metodi che incapsulano la logica di dominio.

```
public class Customer
{
    public Guid Id { get; private set; }
    public string Name { get; private set; }
    public string Email { get; private set; }

    public Customer(string name, string email)
    {
        Id = Guid.NewGuid();
        Name = name;
        Email = email;
    }

    public void UpdateEmail(string newEmail)
    {
        // Validate and update the email
    }
}
```

VALUE OBJECTS

I **Value Objects** sono **oggetti immutabili che non hanno identità e sono caratterizzati dalle loro proprietà**.

Sono utilizzati per rappresentare concetti nel dominio che **non sono entità in sé**, ma **sono comunque essenziali per la modellazione del problema.**

In C#, gli oggetti valore sono implementati come classi o **strutture con proprietà di sola lettura**, confronto di uguaglianza basato sui valori delle proprietà e convalida adeguata del loro stato.

VALUE OBJECTS

```csharp
public class Address
{
    public string Street { get; }
    public string City { get; }
    public string State { get; }
    public string ZipCode { get; }

    public Address(string street,
        string city,
        string state,
        string zipCode)
    {
        Street = street;
        City = city;
        State = state;
        ZipCode = zipCode;
    }

    public bool Equals(Address other)
    {
        if (other is null)
            return false;
        if (ReferenceEquals(this, other))
            return true;
        return Street == other.Street
            && City == other.City
            && State == other.State
            && ZipCode == other.ZipCode;
    }

    public override bool Equals(object obj)
        => Equals(obj as Address);
    public override int GetHashCode()
        => (Street, City, State, ZipCode).GetHashCode();
}
```

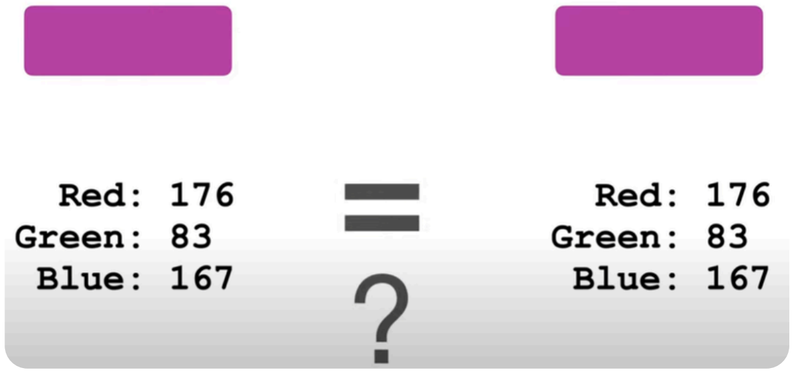
I **Value Objects** sono **oggetti immutabili che non hanno identità e sono caratterizzati dalle loro proprietà**.

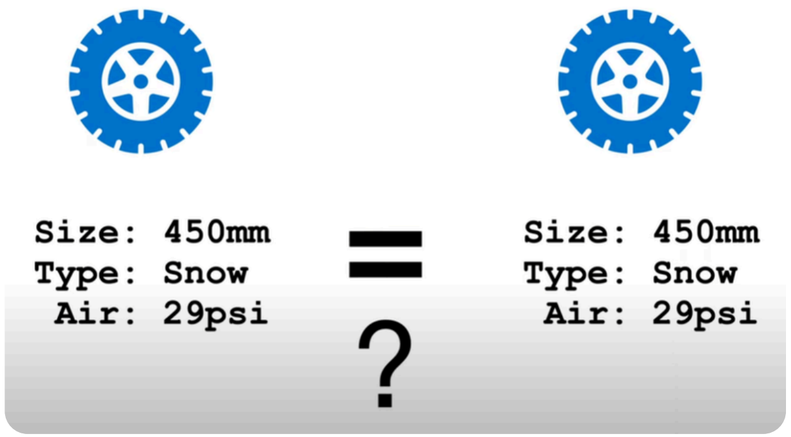
Sono utilizzati per rappresentare concetti nel dominio che **non sono entità in sé**, ma **sono comunque essenziali per la modellazione del problema.**

In C#, gli oggetti valore sono implementati come classi o **strutture con proprietà di sola lettura**, confronto di uguaglianza basato sui valori delle proprietà e convalida adeguata del loro stato.

BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

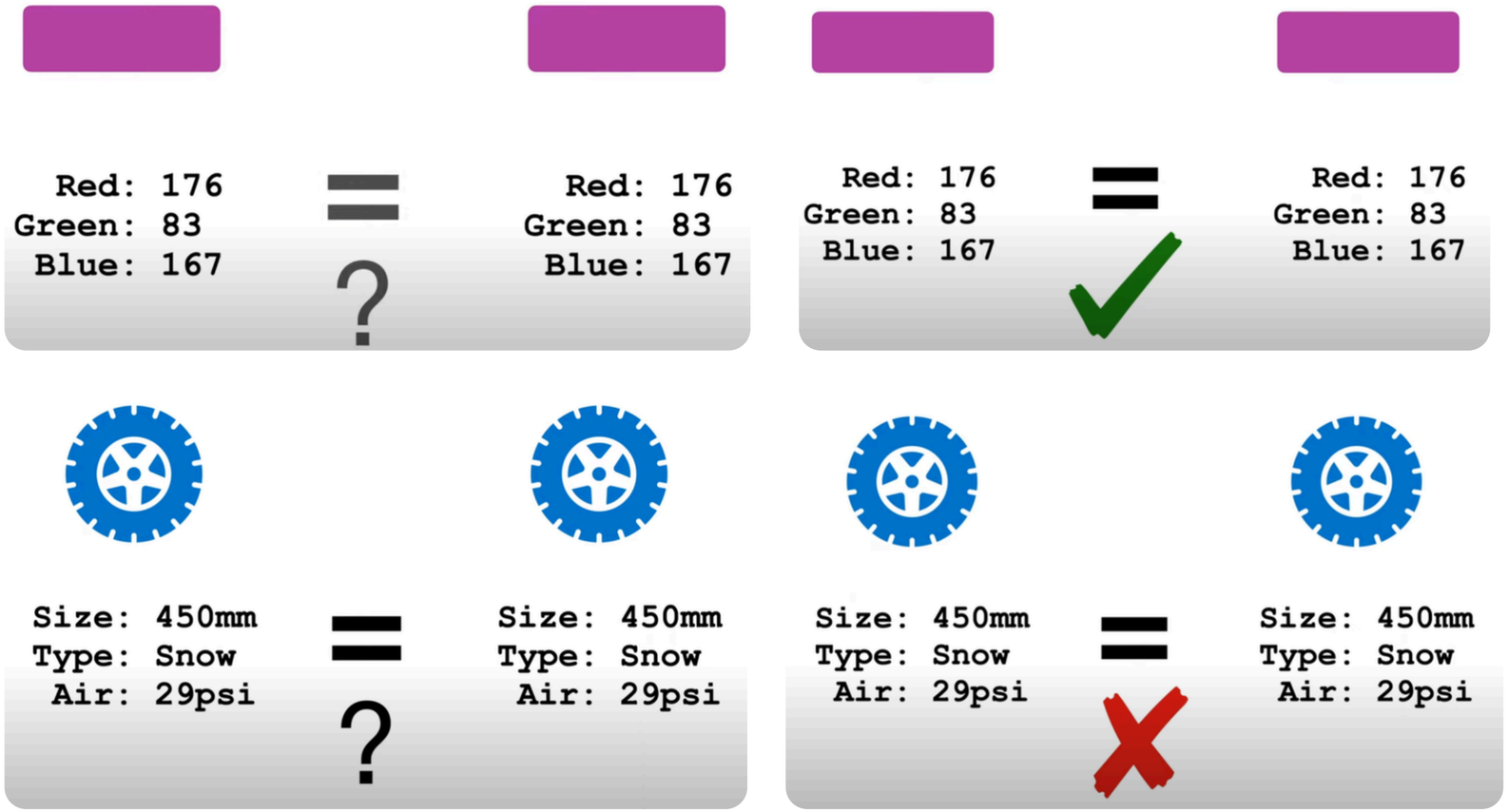
VALUE OBJECTS VS ENTITIES





BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

VALUE OBJECTS VS ENTITIES



TESTING YOUR DOMAIN-DRIVEN DESIGN IMPLEMENTATION

Il testing è un aspetto essenziale di qualsiasi progetto di sviluppo software ed è particolarmente importante per l'implementazione del DDD.

Garantire che la **logica di dominio**, i **casi d'uso** del livello applicativo e i **componenti dell'infrastruttura** funzionino come previsto è fondamentale per il successo dell'applicazione.

Spiegheremo l'importanza dei diversi tipi di test e forniremo esempi di codice C# per testare ogni livello dell'implementazione **DDD** utilizzando **framework** popolari come **xUnit** e **Moq**.

IMPORTANCE OF TESTING

Ci sono tre tipi principali di test da considerare in un progetto DDD:

- **Test unitari**: si concentrano su **singoli componenti o classi**, verificando il comportamento delle **entità di dominio**, degli **oggetti valore** e di altri **elementi costitutivi**.

- **Test di integrazione**: verificano **l'interazione tra diversi componenti**, come i **casi d'uso del livello applicativo** e i **componenti dell'infrastruttura**, quali repository o client di servizi esterni.

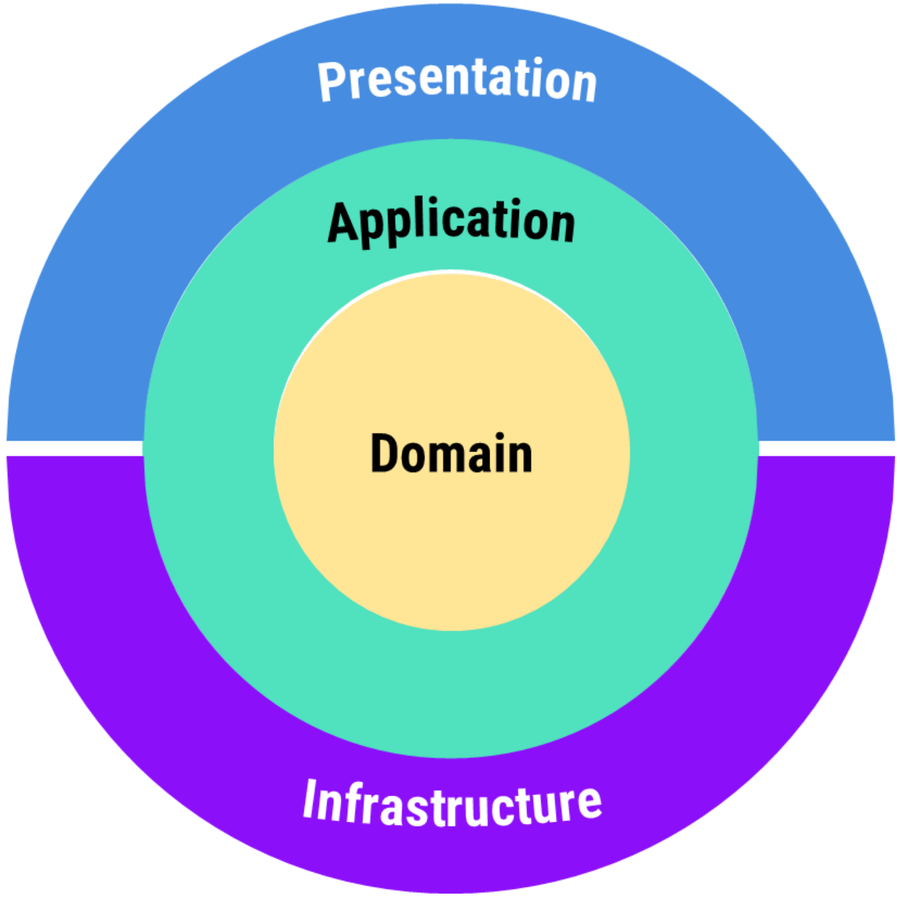
- **Test end-to-end**: convalidano **l'intero sistema**, compresa **l'interfaccia utente e le integrazioni esterne**, assicurando che l'applicazione funzioni correttamente dal **punto di vista dell'utente**.

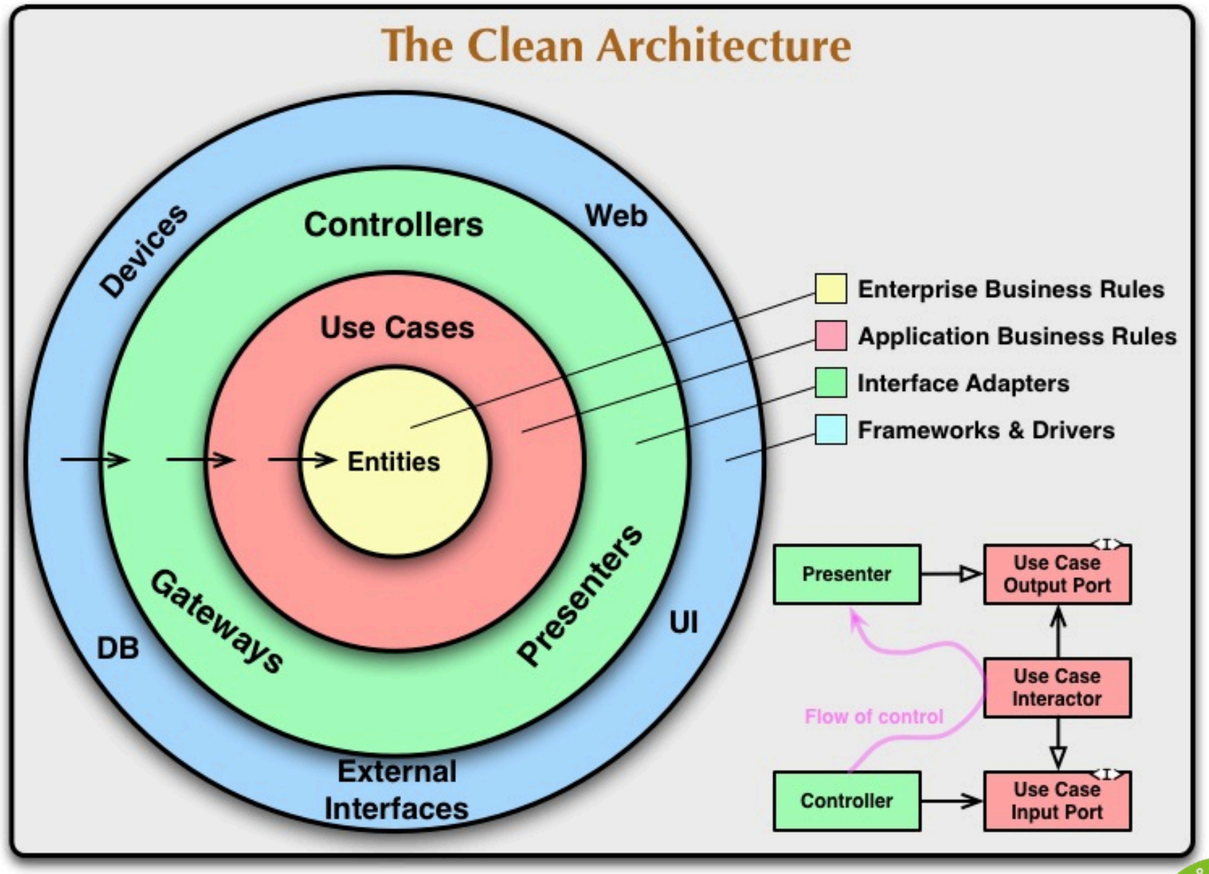
IMPORTANCE OF TESTING





APPLICATION LAYER: IMPLEMENTING USE CASES





BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

APPLICATION LAYER: IMPLEMENTING USE CASES

L'Application Layer nel DDD funge da **ponte tra il Domain Layer e i livelli esterni** (**Outer Layers**) del sistema, come l'interfaccia utente o i servizi esterni.

È responsabile dell'implementazione dei casi d'uso (**Use Cases**), del coordinamento delle attività e della gestione di aspetti trasversali (**Handling Cross-Cutting Concerns**) come la **validazione** e **l'autorizzazione**.

Discuteremo come implementare il livello applicativo in C# e tratteremo concetti chiave come **Commands**, **Queries**, **Handlers** e il **Mediator Pattern**.

Chiama il dominio, ma non il contrario.

APPLICATION LAYER: IMPLEMENTING USE CASES

Nel DDD, **l'Application Layer è responsabile** delle seguenti attività:

- **Orchestrare** l'esecuzione della logica di dominio
- Gestire aspetti trasversali (**Cross-Cutting Concerns**) come la **validazione**, **l'autorizzazione** e la **registrazione**
- Tradurre tra oggetti di dominio (**Domain Objects**) e oggetti di trasferimento dati (**DTO**) o modelli di visualizzazione (**View Models**)

IMPLEMENTING USE CASES

Gli **Use Cases** nell'**Application Layer** possono essere implementati come **Commands** e **Query**.

I **Commands** rappresentano azioni che **modificano lo stato del sistema**,
mentre le **Queries recuperano dati senza causare effetti collaterali**.

In C#, i **Commands** e le **Queries** possono essere implementati come classi, con classi **Handler** corrispondenti responsabili dell'esecuzione del comportamento desiderato.

Gli **Handlers** interagiscono con Domain Objects e servizi, nonché con **componenti dell'infrastruttura** come i **Repository**, per **soddisfare il caso d'uso**.

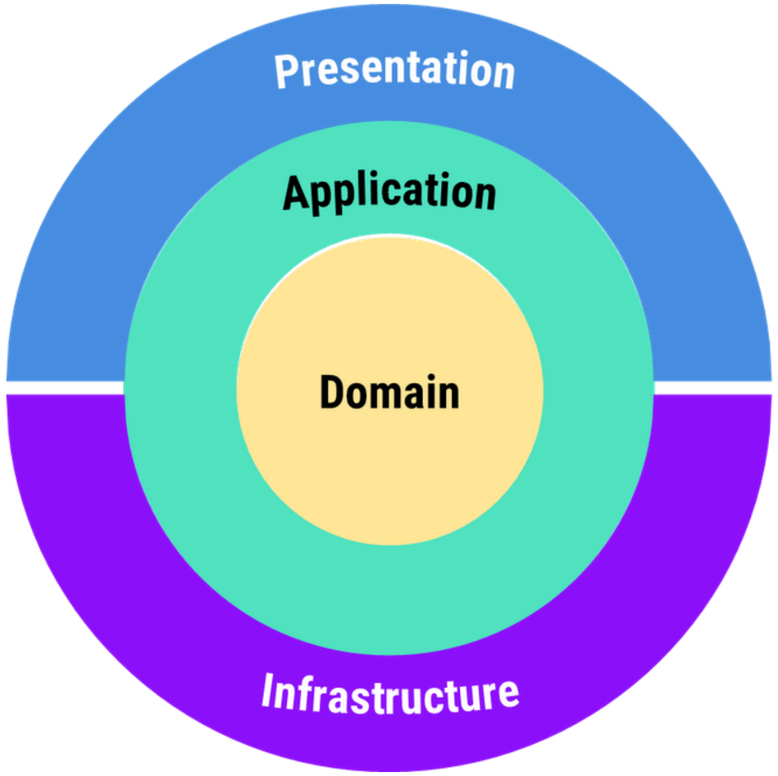
CROSS-CUTTING CONCERNS

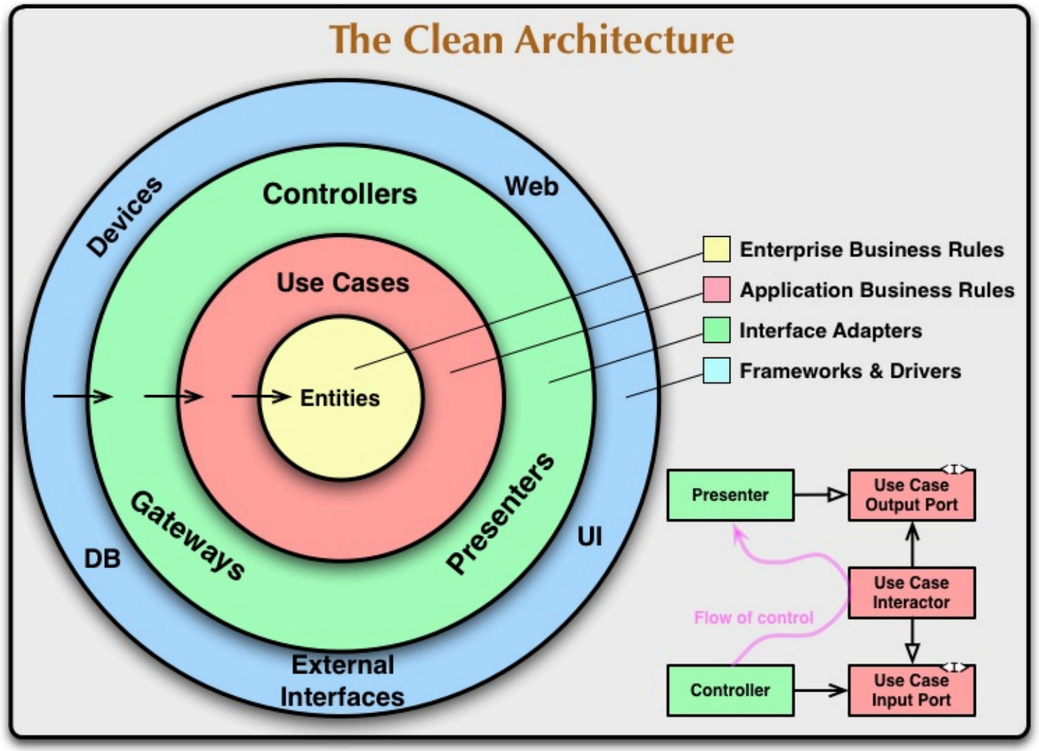
Cross-cutting concerns come la **validazione** e **l'autorizzazione** possono essere gestiti utilizzando middleware, filtri o decoratori nel livello dell'applicazione.

Ad esempio, la convalida dei **Commands** e delle **Queries** può essere eseguita utilizzando la libreria FluentValidation, che consente agli sviluppatori di definire regole di convalida in una classe separata, rendendo il codice più gestibile e testabile.

INFRASTRUCTURE LAYER:

PERSISTENCE AND EXTERNAL SERVICES





BLAISE PASCAL
ISTITUTO TECNICO TECNOLOGICO - CESENA

INFRASTRUCTURE LAYER:

PERSISTENCE AND EXTERNAL SERVICES

Il **Infrastructure Layer** è responsabile della **fornitura di servizi tecnici e implementazioni a supporto dei livelli superiori** del sistema, quali **meccanismi di persistenza**, **integrazioni di servizi esterni** e **configurazione**.

Discuteremo lo scopo dell'Infrastructure Layer in un'implementazione DDD e forniremo esempi di codice C# e best practice per lavorare con la persistenza e i servizi esterni.

PURPOSE OF THE INFRASTRUCTURE LAYER

Il livello infrastrutturale svolge un ruolo cruciale in un'implementazione DDD:

- Implementando la **persistenza** e la **logica di accesso ai dati per gli oggetti di dominio** (domain objects)
- Integrandosi con **servizi esterni**, come code di messaggi (**message queue**) e **API di terze parti**
- Fornendo **servizi tecnici e utilità**, come la **registrazione**, la **memorizzazione nella cache** e la **configurazione**